



University of
Stavanger

Faculty of Science
and Technology

EXAM IN SUBJECT: **DAT650 BLOCKCHAIN TECHNOLOGY**
DATE: **EKSAMEN, NOVEMBER 2024**
DURATION: **4 HOURS**
ALLOWED REMEDIES: **NONE**
THE EXAM CONSISTS OF: **15 EXERCISES ON 8 PAGES**
CONTACT DURING EXAM: **LEANDER JEHL, TLF. 94120764**
REMARKS: None
ATTACHMENTS:

Solution: Note that solutions in the boxes are more solution sketches. Full solutions may include more details and understanding.

Algorithm 1 Reentrancy

Question 1: Solidity (13%)

1:

```
pragma solidity =0.5.11
2: contract SimpleBank {
3:
4:     mapping(address => uint) balances;
5:
6:     // deposit money.
7:     function deposit() public payable {
8:         require(msg.value > 0);
9:         uint balances[msg.sender] = balances[msg.sender] + msg.value;
10:    }
11:
12:    function withdraw(uint value) public {
13:        require(balances[msg.sender] > value);
14:        (bool success) = msg.sender.call.value(value);
15:        if (success) {
16:            balances[msg.sender] = balances[msg.sender]-value;
17:        }
18:    }
19: }
```

Algorithm 2 Ether King Game

```
1: pragma solidity =0.5.11
2: contract EtherGame {
3:
4:     uint public payoutMileStone1 = 3 ether;
5:     uint public mileStone1Reward = 2 ether;
6:     uint public payoutMileStone2 = 5 ether;
7:     uint public mileStone2Reward = 3 ether;
8:     uint public finalMileStone = 10 ether;
9:     uint public finalReward = 5 ether;
10:    bool public finished = false;
11:
12:    mapping(address => uint) redeemableEther;
13:    // Users pay 0.5 ether. At specific milestones, credit their accounts.
14:    function play() public payable {
15:        require(msg.value == 0.5 ether);           // each play is 0.5 ether
16:        uint currentBalance = address(this). balance + msg.value;
17:        // ensure no players after the game has finished
18:        require(!finished);
19:        // if at a milestone, credit the payer's account
20:        if (currentBalance == payoutMileStone1) {
21:            redeemableEther[msg.sender] += mileStone1Reward;
22:        }
23:        else if (currentBalance == payoutMileStone2) {
24:            redeemableEther[msg.sender] += mileStone2Reward;
25:        }
26:        else if (currentBalance == finalMileStone) {
27:            finished = true;
28:            redeemableEther[msg.sender] += finalReward;
29:        }
30:        return;
31:    }
32:
33:    function claimReward() public {
34:        // ensure the game is complete
35:        require(finished);
36:        // ensure there is a reward to give
37:        require(redeemableEther[msg.sender] > 0);
38:        uint transferValue = redeemableEther[msg.sender];
39:        redeemableEther[msg.value] = 0;
40:        msg.sender.send(transferValue);
41:    }
42: }
```

Question 2: Trees (13%)

- (a) (4%) Sketch a Merkle tree with 7 data elements. What data needs to be included in an inclusion proof for the third data element?

Solution: Need hashes h_4 , $h_{1,2}$, and $h_{5,6,7,8}$. Additional position information, left, right, left.

- (b) (3%) If Alice did send some bitcoin to Bob, how can she convince Bob that the payment happened? How can she do this efficiently?

Solution: Alice needs to show Bob a merkle proof showing the transaction is part of a block and block headers for 5 blocks on top.

- (c) (6%) Alice wants to *read* the value of a *owner* variable in a smart-contract deployed on Ethereum. Alice does not run her own Ethereum node. How can she get an answer she trusts. How can this be done efficiently?

Solution: Contracts in Ethereum contain the storage root, i.e. the root of a Merkle tree showing the contracts storage, i.e. all variables. Alice could receive a Merkle proof showing the value of the owner variable. Further, the state of a contract in Ethereum is included in the Storage tree. The root of the tree is included in every block. Alice could receive an inclusions proof for the state of the given smart contract. Finally, Alice should receive the block header, including the storage root, and possibly multiple headers on top, to show confirmation.

Question 3: Unspent transaction output (UTXO) (11%)

- (a) (6%) What are the different elements of a transaction in Bitcoin? How is a transaction validated?

Solution: Transactions include inputs and outputs. Validation checks that

- Inputs reference previously unspent outputs
- Inputs fulfill condition on outputs (i.e. signature with correct key)
- Sum of outputs is less or equal to inputs.

- (b) (2%) How does Bitcoin prevent the following attack:

- Alice wants to send bitcoin to Bob. She issues a transaction for this transfer. Charly intercepts this transaction before it reaches the Bitcoin network. He replaces Bobs public key with his own, submits the new transaction, and thus receives the payment from Alice.

Solution: Alice needs to sign the transaction. The signature includes also the public key of Bob. With the public key replaced, the transaction will no longer be valid, unless Alice creates a new and different signature.

- (c) (3%) Name 3 reasons, why technology like Bitcoin is not suited to be used in everyday payments.

Solution: Throughput, Confirmation time, Cost

Question 4: Proof of work (19%)

- (a) (6%) We had the following definition for a proof of work function:

Def: For an integer d , the proof-of-work (PoW) function with difficulty d takes a data item and returns a nonce (random bits) and a hash value:

$$(h_{PoW}, nonce) = f_{PoW}(Data)$$

The proof of work is valid, if a) h_{PoW} is the hash of the data, concatenated with the nonce

$$h_{PoW} \stackrel{?}{=} H(Data || nonce)$$

and b) the first d bits of h_{PoW} are 0.

What are the shortcomings of this definition and how can it be adjusted?

Solution: The above definition does not allow fine tuning the difficulty. Only allows to make PoW twice or half as difficult.
A better definition for a difficulty is to take d a hexadecimal number and require $h_{PoW} < d$.

- (b) (4%) Assume Alice finds a nonce that allows her to solve the PoW puzzle and create a new block for the Bitcoin blockchain. Alice sends this block to the bitcoin network.

Assume further that Bob is the first node in the network to receive Alices block. Can Bob take the Nonce from Alices block and cash in the block reward himself?

Solution: No. To receive the block rewards, Bob needs to include his address in the coinbase transaction. Thus his block will be different from Alices block and he will most likely need a different nonce.

- (c) (4%) Assume Alice runs a bitcoin miner. After searching for a solution for 10 minutes, Bob finds a block. If Alice stops her initial mining, and instead mines on top of the block that Bob has found, does she lose an advantage? Explain.
- (d) (1%) a proof of work function needs three properties, *Fast Verification*, *Adjustable difficulty*, and *Progress freedom*. Which of the three properties of a PoW function is relevant for the example in Part c.

- (e) (4%) We say that PoW is fair if in the longest chain, the amount of blocks from every miner is proportional to his mining power/hash rate.
- Under which conditions, is a PoW scheme like in Bitcoin not fair? Give an example.
 - Explain one technique to alleviate this problem.

Question 5: Attacks (3%)

- (a) (3%) How can a miner benefit from a selfish-mining attack? Will he mine more blocks?

Solution: The attacker will not mine more blocks, but he causes other blocks to be discarded. This can increase the fraction of blocks in the longest chain, created by the attacker.

Question 6: Proof of Stake (6%)

In PPCoin (Peercoin) a miner identified by *addr*, that has deposited `coin(addr)` can supply the current block, if

$$H(\text{prevBlockHash}||\text{addr}||\text{timeinseconds}) < d_0 \cdot \text{coin}(\text{addr})$$

- Here d_0 is a base difficulty. The probability that a miner with a specific address *addr* can mine the next block is proportional to `coin(addr)`.
 - `timeinseconds` shows time in seconds. Thus, a miner gets a change to submit a solution every second.
- (a) (3%) Give an example, where a PoS-miner can benefit from an attack similar to proof of work.
- (b) (3%) Explain the nothing-at-stake problem. How can that problem be solved?

Question 7: Off-Chain technology (9%)

Off-chain technology includes Payment-Channels, Payment-Channel networks and Commit Chains.

- (a) (5%) What are the advantages and disadvantages or limitations of payment channels, compared to on chain transactions?

Solution: Advantages: Fewer fees, fast confirmation. Disadvantages: Need to lock fees, need to stay online, only single recipient.

- (b) (4%) What are the advantages and disadvantages of payment channels, compared to commit chains?

Solution: Channels: Fast confirmation+ Commit Chains: No registration cost+

Question 8: Ethereum (6%)

How are transaction fees set in Ethereum, and how can a user control, predict, or limit the fees? How can the user set fees to ensure inclusion in the blockchain?

Solution: Based on what code the transaction executes, a cost in Gas is calculated. This can be predicted by pre-executing the transaction, but may change for the actual execution. The fee in Ether is then computed as the gas cost * gas price given by the user. The gas price needs to be set large enough, otherwise, the transaction will not be included in the blockchain. A large gas price makes the transaction being included faster. The user can also give a maximum gas cost. The transaction will not use more gas than this maximum but stop and revert if it reaches this point.

Question 9: Solidity (12%)

The contract given in Algorithm 3 is vulnerable to re-entrancy.

- (a) (4%) Describe how the vulnerability can be exploited by an attacker.
- (b) (4%) Rewrite the `deposit` function to remove the vulnerability. You should still use `call` to send money.
- (c) (4%) What is the advantage of using `call`, rather than `transfer` when sending money. Why is the first vulnerable and the second is not?

Question 10: Ethereum consensus (8%)

How are blocks created in Ethereum 2.0?

Solution: Explain slots and epochs. Mention PoS and that validators need to vote. Mention LMD GHOST gives liveness, says which fork to build on. Mention Casper FFG creates checkpoints.

Algorithm 3 Reentrancy

```
1: pragma solidity =0.5.11
2: contract SimpleBank {
3:
4:     mapping(address => uint) balances;
5:
6:     // deposit money.
7:     function deposit() public payable {
8:         require(msg.value > 0);
9:         uint balances[msg.sender] = balances[msg.sender] + msg.value;
10:    }
11:
12:    function withdraw(uint value) public {
13:        require(balances[msg.sender] > value);
14:        (bool success) = msg.sender.call.value(value);
15:        if (success) {
16:            balances[msg.sender] = balances[msg.sender]-value;
17:        }
18:    }
19: }
```

Question 11: State and transactions (8%)

- (a) (2%) What is recorded on the bitcoin blockchain? Where are balances stored?

Solution: The blockchain only records transactions. Balances and state are stored in memory, computed based on transactions.

- (b) (2%) What is stored in the Ethereum blocks? How is this different from Bitcoin?

Solution: Transactions, state root, receipt root, and other metadata. The blockchain actually contains the root of the state of all smart contracts?

- (c) (4%) Why is the state root important in Ethereum? What can it be used for?

Solution: The state root is used to verify the state of a contract. It can be used to prove the state of a contract, i.e. the value of a variable.

Question 12: Tokens (3%)

- (a) (1%) How is the transfer of a secondary (ERC20) token different from a native token?

Solution: A native token is transferred from A to B by a transaction from A to B. A secondary token is transferred by a transaction from A to the token contract, which then updates the balances of A and B.

- (b) (2%) What is the advantage of a ERC20 token?

Solution: The token is standardized to be traded on exchanges or other markets.

Question 13: Ethereum 2 (2%)

- (a) (1%) What is the function of LMD GHOST in Ethereum 2.0?

Solution: A rule to select which fork to extend or vote for.

- (b) (1%) What is the function of Casper FFG in Ethereum 2.0?

Solution: Creating checkpoints to finalize blocks. Checkpoints are good against long-range attacks.

Question 14: Mixer (2%)

- (a) (1%) What is a mixer?

Solution: A technology to prevent linking of transactions from one account. The ingoing transactions are an anonymity set for outgoing transactions.

- (b) (1%) What is an anonymity set?

Solution: Creating checkpoints to finalize blocks. Checkpoints are good against long-range attacks.

- (c) (3%) How does zero-coin or zero-cash provide a bigger anonymity set than a simple centralized mixer?
- (d) (1%) What is the advantage for normal (law abiding users) of having anonymity in a cryptocurrency?

Question 15: Decentralization (2%)

- (a) (2%) What is decentralization?

Solution: Name dimensions, architecture, governance.

- (b) (2%) If your system is running on many computers, is it decentralized?

Solution: Open question. Allows to talk about governance, participation, ...